13. Stream in Java — Quick Notes

Definition

Stream is: pipeline of functions applied on a source.
Added in: Java 8

Source of Stream

Stream can be created from:

- Collection
- Array
- File lines
- Infinite streams

Example: list.stream()

Stream Structure

Source → Intermediate Operations → Terminal Operation

Example:

```
list.stream()
  .filter(x -> x % 2 == 0)
  .filter(x -> x > 10)
  .forEach(System.out::println);
```

Output: 20 \n 30

Explanation

Part	Type
`list`	Source
`filter()`	Intermediate
`forEach()`	Terminal

Intermediate Operations

Features

- Input → Stream
- Output → Stream
- Lazy execution

Examples:

- filter()
- map()
- sorted()

.filter(x -> x % 2 == 0)

Terminal Operations
Features

- Ends stream pipeline
- Produces final result

Examples:
- forEach()
- count()
- findFirst()

.forEach(System.out::println)

Important Point
Intermediate operations: do NOT execute immediately
Execution starts only when: terminal operation called
This is called: Lazy Execution

Internal Working
Instead of: processing whole collection multiple times
Each element passes through: entire pipeline one by one

Example Flow
```
list.stream()
  .filter(x -> x % 2 == 0)
  .filter(x -> x > 10)
  .forEach(System.out::println);
```

Processing

Element	Result
5	filtered
10	filtered
20	printed
30	printed
8	filtered
7	filtered

findFirst() Optimization

Example:

```
list.stream()
  .filter(x -> x % 2 == 0)
  .filter(x -> x > 10)
  .findFirst();
```

Important

Stops immediately after finding: first matching element
Output: 20

Why Streams Are Efficient?

Because of:

- Lazy execution
- Pipeline processing
- No extra storage

Important Point

Streams: do NOT store data
They are: abstraction over source

Actual storage belongs to:

- List
- Array
- Collection

Parallel Streams

Purpose

Use multiple CPU cores.
Example: list.parallelStream()

Useful For

- Large data
- Parallel processing

Stream Sources

Source	Example
Collection	`list.stream()`
Array	`Arrays.stream(arr)`
File	`Files.lines()`
Generated Stream	`Stream.generate()`

Common Stream Methods

Method	Type
filter()	Intermediate
map()	Intermediate
sorted()	Intermediate
forEach()	Terminal
count()	Terminal
findFirst()	Terminal

Important Quick Points

Topic	Point
Stream	Pipeline of functions
Added in	Java 8
Source	Collection/Array/File
Intermediate Ops	Return Stream
Terminal Ops	End pipeline
Lazy Execution	Executes at terminal operation
Stream Storage	No own storage
parallelStream()	Parallel processing